Rapport Labo 2 - Computer Architectuur

Victor Deforce en Cían Olieslagers

February 24, 2025

1 Probleemstelling

1.1 Opgave

Het doel van de opgave is om race conditions in parallelle berekeningen te analyseren en op te lossen door verschillende implementaties van maximumdetectie te vergelijken. Hierbij worden drie methoden gebruikt:

1. Geneste for lus op de CPU:
   Een sequentiële vergelijking van alle elementen. Dit dient als vergelijking tussen CPU en GPU oplossing.
2. Een GPU kernel met atomic operation:
   Een sequentiële aanpak waarbij een gedeelde variable met atomic instructies worden bijgewerkt.
3. Een GPU kernel met een reduceer-algoritme:
   Een efficiëntere parallelle aanpak die synchronisatie tussen threads gebruikt om race conditions te voorkomen.

2 Implementatie

2.1 For-loop on the CPU

De implementatie zoekt de maximale waarde in een array met een eenvoudige lineaire scan. De functie max_Cpu(int* arr, int size) neemt een array en de grootte als invoer en initialiseert de maximale waarde met arr[0]. Vervolgens doorloopt een for-lus de array en vervangt Maxval als een groter element wordt gevonden.
De tijdcomplexiteit is O(N), wat betekent dat de uitvoeringstijd lineair toeneemt met het aantal elementen

2.2 Atomic Operations

Hier gaan we opnieuw sequentieel te werk. Er wordt met een atomicMax instructie de vergelijking gemaakt tussen het eerste element en het element waarmee de thread id overeenkomt. De atomic instructie kopieert automatisch het grootste element naar het adres van het eerste argument. Eens alle threads hun werk hebben afgerond komt hierdoor het grootste element in de eerste positie. Deze methode heeft dus ook een tijdscomplexiteit van O(n) plus de extra overhead uit het aanroepen van de GPU. We meten zowel de tijdsduur van de kernel als de extra overhead uit geheugen allocatie en het overzetten van de array.

2.3 GPU kernel using reduction

Deze implementatie gebruikt een reductie-algoritme om het maximum in een array efficiënt te bepalen via parallelle verwerking. In plaats van elke thread individueel een variabele bij te werken, combineren threads hun waarden in opeenvolgende reductiestappen, waardoor het aantal bewerkingen exponentieel daalt en race conditions worden voorkomen. Het bepalen van het maximum gebeurt in twee fasen. In de eerste fase bepalen we per reeks van 2048 elementen het maximum met reductie. Elke

reeks heeft dus zijn eigen blok aan threads. Thread i maakt de vergelijking tussen element $i * spil$ en $i * spil + spil/2$ waarbij de spil iedere iteratie verdubbelt en enkel vergelijkingen worden gedaan waarvoor geldt $i * spil + spil/2 < 2048$. Iedere iteratie worden alle threads per blok gesynchroniseerd. Na deze fase worden alle threads met cudaDeviceSynchronize() gesynchroniseerd. In de tweede fase worden op analoge manier de maxima van de verschillende blokken vergeleken. Indien er meer dan 2^{22} elementen in een array zitten zou er nog een derde fase toegevoegd moeten worden, aangezien er daardoor meerdere blocks nodig zijn in tweede fase. En er dus pas een vergelijking gemaakt kan worden tussen die blokken eens alle threads terug gesynchroniseerd zijn.
De tijdcomplexiteit wordt zo teruggebracht van O(N) (sequentiële verwerking) naar $O(log_2(N))$, waardoor de aanpak veel efficiënter is voor grote datasets. Deze methode schaalt goed op GPU's door gebruik te maken van parallelle threadverwerking.

We meten hier ook zowel de tijdsduur van de kernel als de extra overhead uit geheugen allocatie en het overzetten van de array.

3 Observatie

3.1 Tijd vergelijking variërende input size

De uitvoeringstijd van de implementaties varieert afhankelijk van de grootte van de invoer. De CPU- en atomicMax implementaties hebben een lineaire tijdcomplexiteit $O(N)$, terwijl de reduction-implementatie een logaritmische tijdcomplexiteit $O(\log N)$ kan bereiken.

We hebben elke methode 2000 keer laten lopen per array, met lengtes gelijk aan machten van 2 van 2^0 tot 2^{16}. Dit hebben we in boxplots uitgezet in figuur 1 per array lengte. Uit de tijdsduur metingen is het duidelijk dat de cpu het snelst werkt voor arrays met lengte kleiner dan 2^{18} à 2^{19}. Dit is te wijten aan de extra overhead uit het aanroepen van de GPU. Verder zien we weinig onderscheid tussen de meetwaarden die de geheugen allocatie meenemen en deze die dat niet doen. Hieruit kunnen we concluderen dat meeste overhead komt uit het oproepen van de kernel. Daarnaast presteert de atomicMax implementatie beter dan de reduction methode. We denken dat dit komt door de twee kernels die we oproepen in de reduction methode en dat we nog te kleine arrays gebruiken om duidelijk verschil te zien tussen de effecten van een betere tijdscomplexiteit en de overhead. [1]

3.2 Input larger dan 2048

Wanneer de invoergrootte groter is dan 2048, moeten meerdere blokken worden gebruikt, omdat het maximale aantal threads per blok beperkt is tot 1024. Met __syncthreads() kunnen we enkel threads binnen dezelfde blok synchroniseren dus moeten we cudaDeviceSynchronize() gebruiken maar dat zorgt voor extra overhead doordat de kernel tweemaal moet worden opgeroepen. In onze implementatie met reduction, die hierboven is uitgelegd, passen we beide toe op een manier die inputs tot 2^{22} elementen toelaat.

3.3 Calling kernel with less threads than data elements

Indien de kernel minder threads bevat dan het aantal data-elementen (bijvoorbeeld slechts een tiende), moeten threads meerdere iteraties uitvoeren om alle elementen te verwerken. Dit kan worden opgelost door:

- Elke thread meerdere elementen te laten verwerken met een **loop-unrolling**-techniek:

```
int idx = threadIdx.x + blockIdx.x * blockDim.x;
for (int i = idx; i < size; i += blockDim.x) {
    atomicMax(&maxVal, arr[i]);
}
```

- Dit garandeert dat elk element minimaal één keer verwerkt wordt, terwijl het aantal threads beperkt blijft.

[1] Meetwaarden

3.4 Hoe een 2D-array aanpakken?

De aanpak bestaat uit twee stappen:

- Stap 1: Bepaal het maximum per rij dmv reduction.
- Stap 2: Combineer de rijmaxima met reduction om het globale maximum te verkrijgen.

4 Conclusie

Door de vergelijkingen parallel uit te voeren, is de tijdscomplexiteit lager voor een gegeven array met lengte N. Om race conditions te vermijden moeten threads synchroniseren, zowel binnen blocks als over verschillende blokken heen, resulterende in extra overhead. De extra overhead die gepaard komt met het gebruik van GPU zorgt ervoor dat pas bij voldoende grote N de lagere tijdscomplexiteit relevant wordt.

5 Appendix - code

5.1 Code nested for-loop on the CPU

```
#include <cuda_runtime.h>
#include <stdio.h>

int max_Cpu(int* arr, int size) {
    int Maxval = arr[0];
    for (int i = 0; i < size; i++) {
        if (arr[i] > Maxval) {
            Maxval = arr[i];
        }
    }
    return Maxval;
}
int main()
{
    // Oefening 1
    int h_arr[10] = { 10,23,1,4,53,7,69,1,2,4 };

    int max_Value = max_Cpu(h_arr, 10);

    printf("Max value: %d\n", max_Value);  // print for debugging
    return 0;
}
```

5.2 Code Atomic Operation

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <limits.h>

__global__ void max_atomic(int* arr, int* MaxVal, int size) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;  // FIXED index calculation
    if (idx < size) {
        atomicMax(MaxVal, arr[idx]);
    }
}

int main()
{
    // Oefening 2
    int *d_arr;
    int *d_max;
    int h_max = INT_MIN;
    int h_arr[10] = { 10,23,1,4,53,7,64,1,2,4 };


    // Geheugenalloceren
    cudaMalloc(&d_arr, 10 * sizeof(int));
    cudaMalloc(&d_max, sizeof(int));

    cudaMemcpy(d_arr, h_arr, 10 * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_max, &h_max, sizeof(int), cudaMemcpyHostToDevice);

    max_atomic << <1, 10 >> > (d_arr, d_max, 10);
    cudaDeviceSynchronize();

    // Copy Result Back to CPU
    cudaMemcpy(&h_max, d_max, sizeof(int), cudaMemcpyDeviceToHost);
    printf("GPU Max Value: %d\n", h_max);  // FIXED printf

    // Free Memory
    cudaFree(d_arr);
    cudaFree(d_max);

    return 0;
}
```

5.3 Code GPU kernel using reduction

```
__global__ void maxReductionSyncKernel(int *array, int size) {
  int roof = size>2048 ? 2*blockDim.x : size;
  for(int spil = 2; spil<=roof; spil=spil*2){
    int idx = (threadIdx.x * spil + blockIdx.x * blockDim.x);
    if (idx+spil/2<((1+blockIdx.x) * 2 * blockDim.x)) {
      if (array[idx] < array[idx + spil / 2]) {
        array[idx] = array[idx + spil / 2];
      }
    }
    __syncthreads();
  }
}
__global__ void maxReductionRestKernel(int *array, int size) {
  for(int spil = 4096; spil<=size; spil=spil*2){
    int idx = (threadIdx.x + blockIdx.x * blockDim.x)*spil;
    if (idx+spil/2<size) {
      if (array[idx] < array[idx + spil / 2]) {
        array[idx] = array[idx + spil / 2];
      }
    }
    __syncthreads();
  }
}


int main()

{
    for (int j = 0; j < iterations; j++) {
      int* h_a = new int[size];
      for(int i = 0; i < size; i++){
        h_a[i] = 0;
      }
      h_a[size / 3] = std::rand();
      int *d_a;
      int nBlocks = (size>2048) ? (size / 2048) : 1;
      auto start = std::chrono::steady_clock::now();

      cudaMalloc((void **)&d_a, size * sizeof(int));
      cudaMemcpy(d_a, h_a, size * sizeof(int), cudaMemcpyHostToDevice);

      maxReductionSyncKernel<<<nBlocks, (size/(2*nBlocks))>>>(d_a, size);
      cudaDeviceSynchronize();
      maxReductionRestKernel<<<1, nBlocks>>>(d_a, size);

      cudaMemcpy(h_a, d_a, size * sizeof(int), cudaMemcpyDeviceToHost);

      long duration = std::chrono::duration_cast<std::chrono::microseconds>(
                          std::chrono::steady_clock::now() - start)
                          .count();

      if (h_a[0] != h_a[size / 3]) {
        printf("size: %i\n", size);
        printf("%i - %i\n", h_a[0], h_a[size / 3]);
        for(int k = 0; k< size; k=k+1024)
```

```
            printf("%i ", h_a[k]);
          exit(1);
        }
        cudaFree(d_a);
        fprintf(file, "%lu,", duration);
        free(h_a);
    }
    return 0;
}
```

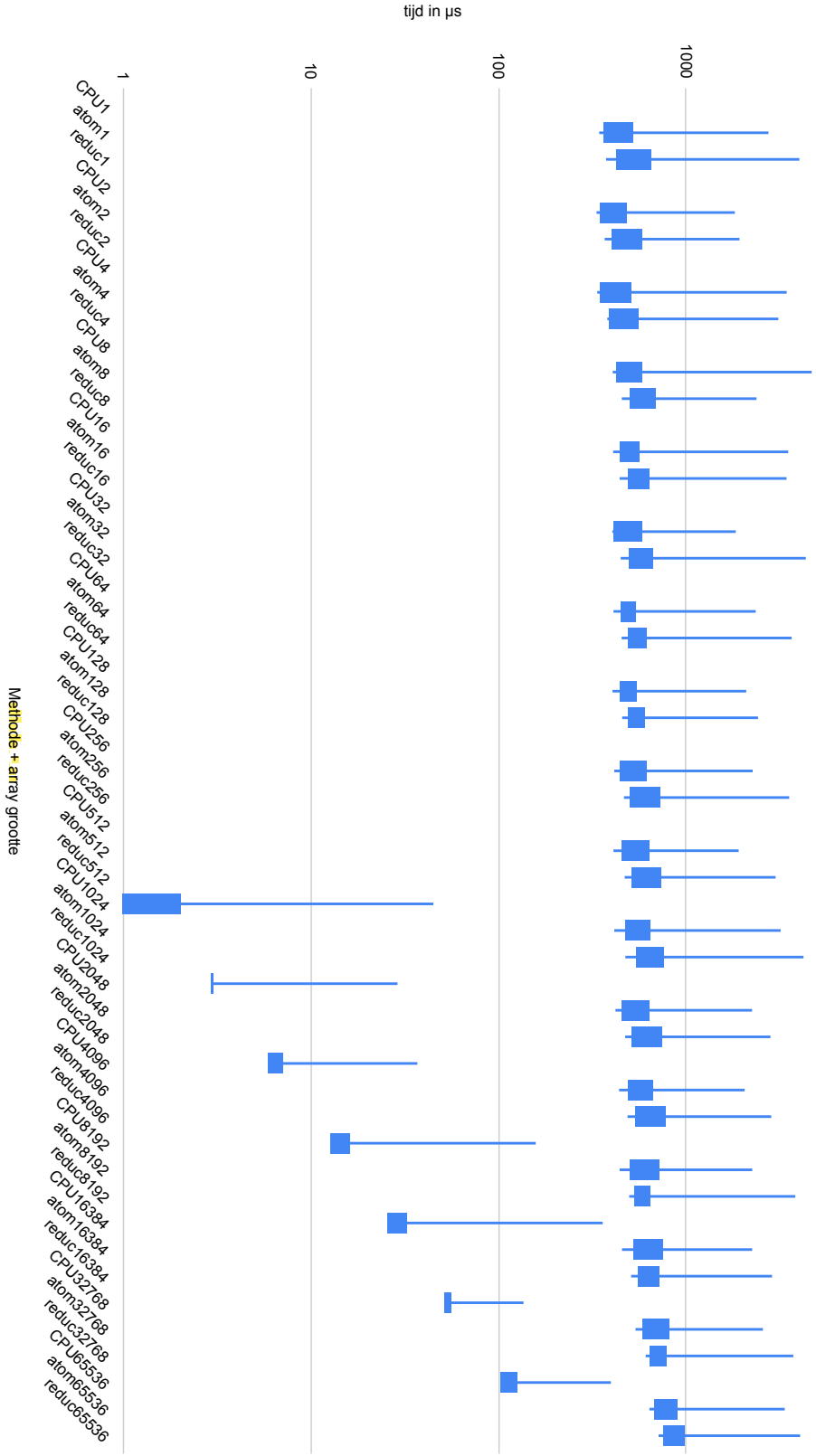


Figure 1: Meetwaarden

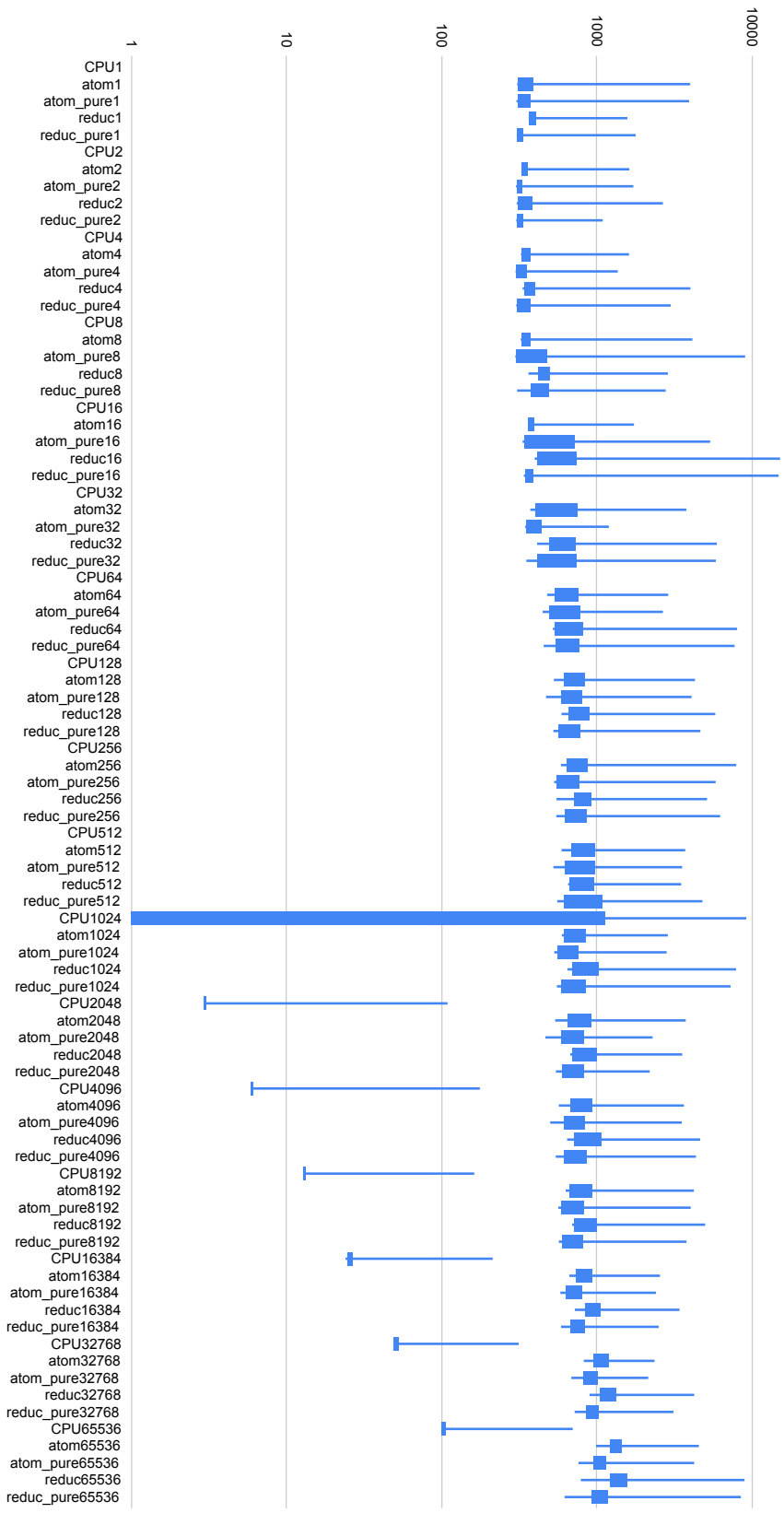


Figure 2: Meetwaarden met onderscheid of geheugen allocatie is meegenomen of niet (pure)